diffmrc Manual

Reference, methods, and verification records

Version 0.3

The reference document for the diffmrc package: installation and configuration, the API surface, protocol details, the design of the differentiable chain and its second derivatives, the verification records, benchmark results, and the development architecture. The companion *Tutorial* covers first use.

Contents

I User reference **5**

1 Overview and package layout **6**
1.1 Design principles . . . 6
1.2 Package layout . . . 6
1.3 How the documents are organized . . . 7

2 Installation and configuration **8**
2.1 Supported environment . . . 8
2.2 Runtime configuration . . . 8
2.3 Verifying the installation . . . 8
2.4 Optional: running the figures . . . 9
2.5 Known environment friction . . . 9

3 API reference **10**
3.1 core: the numerical kernel . . . 10
3.2 analysis: NumPy only . . . 11
3.3 policy: selection rules . . . 11
3.4 driver: the differentiable pipelines . . . 12
3.5 cli . . . 12

4 Protocols and options **13**
4.1 Orbital-point convergence . . . 13
4.2 State averaging . . . 13
4.3 Active-space windows . . . 14
4.4 The e2_scale switch . . . 14
4.5 Differentiation settings . . . 14
4.6 MC-PDFT on-top functionals (tPBE, tPBE0) . . . 14

II Methods and records **16**

5 The differentiable chain **17**
5.1 Parameterization . . . 17
5.2 The two factory families, and why . . . 17
5.3 Inside the chain . . . 17
5.4 What is verified, and against what . . . 18

6 Second derivatives **19**
6.1 The nuclear Hessian by implicit differentiation . . . 19
6.2 The second-order stages never materialize the Hessian . . . 19
6.3 Degenerate pairs and the derivative basis . . . 19
6.4 From the Hessian to frequencies, and one cautionary record . . . 20

7 Verification records **21**
7.1 Energy chain . . . 21
7.2 First derivatives . . . 21
7.3 Second derivatives . . . 22
7.4 External-data cross-checks . . . 22
7.5 The layer contracts . . . 22

8 Benchmarks **23**
8.1 Ground-state line (M5-A) . . . 23
8.2 Excited-state line (QUEST subset) . . . 23
8.3 Selection-protocol cross-check (DVS-APC) . . . 24
8.4 What is not benchmarked . . . 24

9 Known limitations **25**
9.1 Method-family limits . . . 25
9.2 Implementation limits . . . 25
9.2.1 Degenerate pairs at a stationary point . . . 26
9.3 Protocol limits . . . 26

III Development **27**

10 Architecture and contracts **28**
10.1 The six layers . . . 28
10.2 Contracts are tests . . . 28
10.3 How the test suite is organized . . . 28
10.4 Development workflow . . . 29

11 Reproducing the paper **30**
11.1 Environment . . . 30
11.2 The paper's figures and tables . . . 30
11.3 The later result lines . . . 31
11.4 Building this documentation . . . 31

IV Acceleration and performance **32**

12 Resolution of identity (RI/DF) **33**
12.1 The approximation . . . 33
12.2 API reference . . . 33
12.3 Axis and sign conventions . . . 35
12.4 Error budget . . . 35
12.5 Memory . . . 35
12.6 Boundaries . . . 36
12.7 A standalone example . . . 36

13 Performance and numerical hygiene **38**
13.1 MO integral transforms: four sequential contractions . . . 38
13.1.1 Why the one-shot form is not used . . . 38
13.1.2 Accuracy on the release tree . . . 39
13.2 The S_z-blocked CI solver . . . 39
13.2.1 Memory and time at two determinant counts . . . 39
13.3 Automatic solver selection . . . 40

13.3.1 What the switch leaves invariant 40
13.3.2 The small-system penalty the default avoids 41
13.4 Reference arrays as jit arguments 41
13.5 Repeated core folding in the CI matrix–vector product 42
13.6 Choosing among the options 42

V New in version 0.3 **43**

14 New modules in version 0.3 **44**
14.1 Relaxed NEVPT2 nuclear gradients 44
14.2 Coupled CASPT2 46
14.3 Valence-level SOMF for effective-core potentials 47
14.4 RIJONX SCF 48
14.5 Scalar DF-X2C CASSCF 50
14.6 Validation scope and boundaries 51

Part I

User reference

Chapter 1

Overview and package layout

1.1 Design principles

Three commitments shape the package, and most of its structure follows from them.

The first is that derivatives are never written by hand. Every nuclear gradient and second derivative is obtained by differentiating the energy code path with JAX. This is what makes the CASSCF, state-averaged CASSCF, and NEVPT2 layers share one implementation of the physics: a correction to the energy expression propagates to all of its derivatives without a separate derivation.

The second is that the package is layered, and the layer boundaries are tested. The kernel cannot import the layers above it; the analysis layer cannot import the kernel at all. The contract is enforced in the test suite (see Chapter 10), so a violation is a failing test rather than a matter of review discipline.

The third is that convergence is reported, not assumed. Optimizers return the gradient norm at the point they stopped at, and the package distinguishes "the optimizer stopped improving" from "the point is stationary." Callers are expected to carry that distinction into whatever they claim about a result.

1.2 Package layout

```
src/diffmrc/
  core/       numerical kernel: rotations, CASCI, CASSCF, NEVPT2,
              stationary points, the frozen source of the chain
  analysis/   post-processing that reads converged objects: entropies,
              mutual information, spin and state identification,
              orbital identity and frontier windows; NumPy only
  policy/     selection rules over analysis output: composition,
              entropy-based and ranked orbital selection, verification
  driver/     composition layer: assembles the kernel into the
              differentiable SCF -> CASSCF -> NEVPT2 pipelines,
              orbital-point convergence, frozen-geometry factories
  report/     result objects and rendering
  cli/        the command-line entry point
```

The dependency direction runs downward only: `core` at the bottom, `cli` at the top. A user script may import any layer; the layers themselves may not import upward. This is what allows the analysis layer to be tested and used with nothing but NumPy installed.

1.3 How the documents are organized

This manual is the reference. Part I covers installation, the API surface, and the protocol details that the code reports but does not enforce. Part II carries the methods: the design of the differentiable chain, the second-derivative construction, the verification records against independent implementations, the benchmark results, and the consolidated limitation list. Part III is for development: layer contracts, the test architecture, and the reproduction steps for the paper.

The companion *Tutorial* is the on-ramp: installation, a first CASSCF gradient, state-averaged workflows, active-space selection, and a full benchmark walkthrough, each with runnable code.

Chapter 2

Installation and configuration

2.1 Supported environment

The package is developed and verified on Linux and Windows with CPython 3.11, on CPU. The Tutorial (Chapter 2) gives the minimal recipe; this chapter records the constraints behind it.

Dependency	Constraint	Why it is pinned
Python	≥ 3.10	`int.bit_count` in the determinant tools
numpy	≥ 1.24	analysis layer, reference arithmetic
scipy	≥ 1.10	L-BFGS-B and trust-region stages
pyscfad	$\geq 0.3.3$	provides the traceable SCF and integral path
pyscf	≥ 2.5	reference energies and comparisons
jax	≥ 0.4	array layer; the exact pin is pyscfad's
matplotlib	any recent	figure sources only

`pyscfad` pins a specific JAX version, which is why the package should live in its own environment. Mixing it into an environment with a different JAX pin produces failures that look like numerical bugs and are not.

2.2 Runtime configuration

Three settings are load-bearing.

`jax_enable_x64 = True`. The package is validated in double precision. With the default single precision the errors are not loud: gradients lose several digits and some convergence gates become unreachable, while energies look plausible.

`JAX_PLATFORMS = cpu`. The CPU platform is the verified configuration. On a machine with a CUDA device, leaving the platform unset selects the GPU, where the solver path used by the orbital canonicalization raises an error unrelated to the chemistry. Set the variable before importing JAX; setting it afterwards has no effect.

Thread environment variables. The driver assumes it owns the thread budget it was given. Setting the five BLAS thread variables keeps multiple concurrent calculations from oversubscribing a node; their absence does not corrupt results, it makes timing meaningless and slows everything.

2.3 Verifying the installation

```
pytest tests/ -v
lint-imports        # or: pytest tests/test_layer_contracts.py
```

The suite is the primary artifact to look at: it contains the first-order and second-order derivative checks against finite differences, the cross-validation against PySCF, and the layer contracts. A passing suite on your machine is the intended proof that the environment is sound.

2.4 Optional: running the figures

The figure sources under `figures/` import matplotlib and read JSON results produced by the scripts; they require no additional configuration. They are not part of the package tests.

2.5 Known environment friction

- The first evaluation of a freshly built node of the chain triggers XLA compilation that can take minutes. This is expected; a stopwatch on a single evaluation will mislead.
- pyscfad's `Mole` objects cannot cross a `jax.jit` boundary. The driver layer offers frozen-geometry factories for jitted contexts; the factory docstrings state which is which.
- On shared nodes, leave headroom for the owner of the other processes. The memory profile of this package is dominated by compilation- and evaluation-time buffers of the differentiated chain, not by the integrals, and it grows steeply with the number of rotation parameters.

Chapter 3

API reference

This chapter lists the entry points a user is expected to reach for, layer by layer. It is a map, not a transcription: the authoritative signature of every function is its docstring in the source, and the module-level `__all__` lists are the canonical inventories. Entries marked *stable* have been exercised by the verification records in Chapter 7; the rest follow the same conventions but have lighter coverage.

3.1 core: the numerical kernel

Entry point	Purpose
`rotation.block_rotation(t, nmo, pairs)`	rotation matrix from the parameter vector
`orblib.orbital_pairs(ncore, nact, nmo)`	the non-redundant rotation pairs
`casci.cs_casci(...)`	CASCI energies; state-averaged branch with `nroots`, `return_ci`
`optimizer.optimize_theta_scipy(...)`	L-BFGS-B stage
`optimizer.optimize_theta_hand(...)`	the fallback when scipy raises
`grad.make_energy_fn / make_grad_fn`	energy and nuclear-gradient callables
`grad.solve_theta_star(...)`	the two-stage orbital optimization
`grad.nuclear_gradient(mol, grad_fn)`	the differentiated gradient
`stationary.refine_stationary_point(...)`	damped-Newton refinement (Newton–CG step)
`stationary.curvature_report(H)`	the saddle-point check
`stationary.StopReason / StationaryPoint`	convergence verdict types
`nevpt2.impl.cas_dets_for2(...)`	determinant bitmaps of the active space
`nevpt2.impl.canonicalize_jax(...)`	orbital canonicalization on the differentiated path
`nevpt2.impl.nevpt2_e2_full2(...)`	the frozen NEVPT2 contraction
`mcpdft.w_sigma / pi_active`	on-top densities: spin 1-RDM blocks and the factorized Π
`mcpdft.eot_energy / mcpdft_eot`	the on-top energy, kernel injected (tPBE)
`mcpdft.pbe_exc / pbe_eval_xc`	the closed-form PBE kernel (jit-safe)
`mcpdft.exchange_energy(...)`	the hybridś HF-like exchange term (diagnostic)

The kernel is frozen: later layers build on it and corrections to it go through the verification records, not through ad-hoc edits.

3.2 analysis: NumPy only

Entry point	Purpose
`states.state_groups / select_groups`	roots to physical states; state selection
`states.spin_square / det_abs_sz`	exact $\langle S^2 \rangle$ and $\lvert M \rvert$ of determinants
`orbitals.frontier_report / classify_orbital`	frontier identities; window suggestion
`orbitals.heavy_atoms / p_axis_profile`	atom keys; per-axis p profiles
`ci.occupations_from_ci / entropy_from_ci`	occupations and entropies from a CI vector
`ci.pair_density / mutual_information`	two-orbital reductions and their correlations
`rdm.entropy_from_rdms`	the density-matrix route to the same quantities
`diagnostics.is_single_reference / zs1`	multireference diagnostics
`plateau.detect_plateaus`	entropic plateaus of an orbital set

This layer imports nothing from the kernel; it operates on the determinant-bitmap convention and on arrays, and its tests run with NumPy alone.

3.3 policy: selection rules

The policy layer turns analysis output into selections: composition-based rules, entropy-based ordering, the ranked orbital selection with its singly-occupied and virtual-removal options, and the verification helpers. The module names are the inventory (`policy.composition`, `policy.entropy`, `policy.ranking`, `policy.rules`, and their siblings); the docstrings state the conventions each rule follows.

3.4 driver: the differentiable pipelines

Entry point	Purpose
`nevpt2.make_nevpt2_joint_fn(...)`	(mol, θ) SCF→CASSCF→NEVPT2, geometry-differentiable
`nevpt2.make_nevpt2_theta_fn(...)`	frozen geometry, θ only; jit-safe
`nevpt2.make_nevpt2_sa_fn(...)`	state-averaged joint factory
`nevpt2.make_nevpt2_sa_theta_fn(...)`	state-averaged, frozen geometry, jit-safe
`nevpt2.sa_cas_solution(...)`	state-averaged CAS solution and CI vectors
`nevpt2.sa_nevpt2_state_table(out, dets)`	grouped physical-state table
`nevpt2.sa_vertical_excitations(states)`	per-state vertical excitation energies
`kernel.mcpdft_energy_function(...)`	the differentiable one-step tPBE chain (`hyb` for tPBE0)
`kernel.mcpdft_terms_function(...)`	the same chain with its four energy components
`nevpt2.sa_mcpdft_table(...)`	SA-CASSCF + per-state one-step on-top column, explicit window
`nevpt2.sa_mcpdft_table_from_mol(...)`	the same with the frontier window rules
`dvs.dvs_scan(mol, ...)`	the (lambda, N) DVS candidate scan and selection
`kernel.frozen_grid(mol)`	the frozen quadrature (AO values, gradients, weights)
`orbitals.converge_orbital_point(E_theta, npar)`	two-stage convergence with refinement and polish
`orbitals.hessp_of(E_theta)`	Hessian-vector product for the trust-region stage

The two factory families exist because of a pyscfad constraint: the `Mole`-based factories cannot be traced inside `jax.jit`, and the theta-only factories are built for exactly that context. When in doubt about which to call, ask whether the geometry is fixed; if it is, use the theta-only factory.

3.5 cli

`python -m diffmrc.cli` runs an end-to-end calculation from a geometry file; `--help` lists the options. The command-line entry is the least settled surface of the package.

Chapter 4

Protocols and options

The package reports convergence and protocol evidence; it does not enforce it. This chapter collects the settings that decide whether a number is trustworthy, with the values the benchmark records were produced under. Where a default can be changed by a caller, the change belongs next to the number it produced, because several of the gates below are reachable at more than one tolerance and the results differ at the margins.

4.1 Orbital-point convergence

The two-stage protocol separates two different failure modes: the optimizer that can no longer lower the energy, and a point that is not stationary. The stages and their defaults:

Setting	Value	Meaning
`optimize_tol`	10^{-6}	L-BFGS-B target; loose on purpose
`refine_target`	10^{-11}	the Newton target, and the verdict
`max_steps`	2000	L-BFGS-B iteration cap
`refine_max_iter`	80	damped-Newton iteration cap
`_CG_tol` / `_CG_maxiter`	10^{-6} / 40	Newton–CG budget for the step solve

The verdict is the gradient norm at the point the chain stopped at, classified by the stop reason:

`StopReason`	Reading
`CONVERGED`	gradient below the refine target
`STALLED_GRADIENT`	a step no longer reduced the gradient
`GUARD_STOPPED`	no damped step lowered the energy
`ITERATION_CAP`	budget exhausted; re-check before claiming convergence

Two details of the Newton stage are load-bearing and easy to lose when reimplementing. The Hessian enters only through Hessian-vector products — the explicit `jax.hessian` broadcasts every intermediate over n_{par}^2 tangent pairs and was killed by a 402 GB allocation at $n_{\mathrm{par}} \approx 500$ — and the downhill acceptance test is not optional: an unguarded refinement moved an energy from -76.07 to $+1.04$ Ha in this project's history and printed no error.

4.2 State averaging

The state-averaged factories take `nroots` and `state_weights`; the weights are normalized and must have one entry per root. The measured effect of between-group equal weights (the second pass) on the reported excitation energies stayed within a few tenths of an eV for the benchmark

systems, while the second pass usually tightens the final orbital gradient by orders of magnitude. The benchmark records quote which pass each number came from.

Two root-table facts belong with every state-averaged number. The eigensolver returns roots, so a triplet travels as three degenerate roots; and a per-component NEVPT2 correction can be undefined (`nan`) for a pure $|M| = 1$ component, in which case the group's representative is the lowest component with a finite value. `sa_nevpt2_state_table` performs both bookkeeping and returns the evidence.

4.3 Active-space windows

The frontier-window rules are stated in the Tutorial (its chapter on choosing an active space). The part that belongs to a protocol table is the bookkeeping: the electron count follows the window,

$$n_{\mathrm{ele}} = N_{\mathrm{ele}} - 2\, n_{\mathrm{core}}, \tag{4.1}$$

and a window that widens to three orbitals for the linear two-pi and amide patterns therefore carries four active electrons, not two. A hard-coded two here produces a wrong ground state that the energy alone will not flag.

4.4 The e2_scale switch

Both NEVPT2 factories accept `e2_scale`. With `e2_scale = 0` the perturbation correction is switched off and the call reduces to the CASSCF energy of the same chain — the identity used by the test suite to pin the joint and state-averaged factories against the validated single-root path. With the default value of 1.0 the correction is included. There is no other intermediate value with a physical meaning; the switch exists for verification.

4.5 Differentiation settings

The theta-only factories are safe inside `jax.jit`; the `Mole`-based ones are not, and the failure mode is a tracing error at compile time rather than a wrong number. The second-order paths (the refinement, the trust-region polish) differentiate the gradient with forward-mode `jvp` calls; a caller wrapping the whole convergence in `jit` should expect the compile of those nodes to dominate the first evaluation.

4.6 MC-PDFT on-top functionals (tPBE, tPBE0)

The reported energy of `--method mcpdft` is

$$E_{\text{MC-PDFT}} \;=\; E_{\text{mcwfn}} \;+\; E_{ot}, \qquad E_{\text{mcwfn}} \;=\; V_{nn} + \langle h|D\rangle + \tfrac{1}{2}\,\langle J[D]\,|\,D\rangle, \tag{4.2}$$

with D the *full* (core plus active) one-body density and E_{ot} the on-top functional of the translated spin densities, evaluated on a frozen quadrature grid. The conventions that the validation had to pin down, and that a reimplementation must reproduce, live in `core.mcpdft`:

- the translation uses gradient rows $\rho'_\sigma = (\nabla\rho/2)(1 \pm \zeta)$ with **no** $\nabla\zeta$ term (Li Manni et al. 2014; the engine's `get_rho_translated`);
- the on-top pair density is the convention-free factorized form $\Pi = \frac{1}{2}\sum \phi^4 (A - B_{ss}) + (\rho_f\rho_f - \rho_a\rho_a)$, with A the M-block product identical to the stack's spin-summed 2-RDM mechanism;

- the xc kernel is the package's own closed-form PBE (`core.mcpdft.pbe_exc`), because the pyscfad/libxc wrapper cannot be traced under `jit`; it matches libxc to 2×10^{-15} in the physical domain and deviates only below $\rho \sim 10^{-13}$ (libxc's internal guards), where the grid weights put the integral contribution below 10^{-25}.

The frozen grid. The quadrature is built once and shared by every evaluation, so geometry derivatives omit the grid-velocity term. The calibration on CO/6-31G bounds that term at 7.3×10^{-5} Eh/bohr at grid level 3, falling to 2.5×10^{-5} at level 4 – three orders of magnitude below the wavefunction response it rides on.

One-step protocol and the relaxed variant. The orbitals are the CASSCF-driven ones and the on-top energy is evaluated once; this is the DVS paper's protocol. The orbital point is *not* stationary for the on-top functional (measured $|\nabla E| \approx 3 \times 10^{-3}$ there). The theta chain can also minimize the on-top energy itself, which converges to a fully relaxed stationary point below the one-step value by 3.0×10^{-4} Eh on H_2/6-31G; the two numbers answer different questions and the reports say which one they carry.

The tPBE0 hybrid. `--method mcpdft0` is the 0.25 hybrid. Its engine decomposition is $E = E_{\text{classical}} + \lambda(E_x + E_c) + (1-\lambda)E_{ot}$ with $\lambda = 0.25$, E_x the HF-like exchange of the full density and E_c the active-space cumulant contraction. The standard RDM split gives $E_x + E_c = E_{\text{CASSCF}} - E_{\text{classical}}$, so at the one-step orbitals the hybrid is the convex combination $E_{\text{tPBE0}} = 0.75\, E_{\text{tPBE}} + 0.25\, E_{\text{CASSCF}}$; the chain implements exactly this, with no 2-RDM convention in the path.

Verification anchors. Against `pyscf.mcpdft`: H_2/STO-3G reproduces $e_{tot}/e_{ot}/e_{mcwfn}$ to 10^{-16} (golden -1.156628479816; the tPBE0 value -1.151792318484 to 9×10^{-16}), H_2O/STO-3G to 1.6×10^{-9}, and H_2/6-31G to 1.3×10^{-8} – the engine's own FCI convergence floor, measured as its Davidson vector differing from a full block decomposition by $\sim 10^{-7}$. The rotation-chart AD gradient agrees with finite differences to 10^{-10}, and the fully relaxed optimization reaches $|\nabla| = 1.4 \times 10^{-14}$.

The (lambda, N) DVS scan. `driver.dvs.dvs_scan` builds the candidate orbital sets of the DVS construction ($G(\lambda) = F - \lambda K$ diagonalization, the exact t369 reproduction), picks windows with the APC chooser (pyscf's `mcscf.apc`, the authors' implementation), and carries each candidate through the one-step columns; the selection is the lowest mean on-top energy with the runner-up gap reported. Windows whose determinant count exceeds `max_ndet` (default 2000) are refused with the count recorded: the in-stack CI is a full determinant decomposition, so the paper's ZnH/CuCl $(12,12)/(14,14)$ windows stay engine-side by design.

Part II

Methods and records

Chapter 5

The differentiable chain

This chapter describes how the package makes a multireference energy differentiable, and records the numerical facts that shaped the design. It is the software-side companion to the methods paper; the derivations live there, the implementation constraints live here.

5.1 Parameterization

Orbital variation enters through a rotation parameter vector θ: the occupied–virtual mixing is generated by the non-redundant pair list of `orblib.orbital_pairs`, and `rotation.block_rotation` turns θ into the rotation matrix applied to the reference orbitals. Three pair families are included: core–active, core–virtual, and active–virtual, for every orbital outside the active set.

The active–active rotations are omitted deliberately: they are redundant against the CI step, which is complete inside the active space. The core–virtual family is *not* omittable. A parameterization that drops it stops at a point that is not stationary; the defect was measured at the level of a few hundredths of a kcal/mol on a test system — small, systematic, and invisible without an independent reference. The pair list is the contract; do not prune it for speed.

5.2 The two factory families, and why

Every differentiable entry point comes in two forms.

The *joint* factories (`make_nevpt2_joint_fn` and its state-averaged sibling) take the molecule and θ and run the reference SCF inside the call. They are the geometry-differentiable path: the nuclear coordinates enter through the integrals, and JAX differentiates the whole composition.

The *theta-only* factories (`make_nevpt2_theta_fn`, `make_nevpt2_sa_theta_fn`) compute the reference SCF and the integrals once, at construction, and return a callable of θ alone. They exist for one hard reason: this build of pyscfad cannot trace its SCF inside `jax.jit`, and the orbital optimizer wraps its objective in `jit`. A joint factory called from inside the optimizer raises a tracing error; the theta-only factory is the supported path there. The two families funnel through one shared component builder, so they cannot drift apart numerically — the test suite pins them against each other at fixed geometry.

5.3 Inside the chain

A single evaluation composes four numerically delicate pieces.

The MO integral transform runs in four contraction steps. The four-operand form of the two-electron transform is numerically weaker on XLA at large sizes, and the discrepancy is not cosmetic: on an 96^4 integral array the one-shot form showed absolute errors around 10^{-6}, which the near-degenerate gaps of a linear molecule's generalized Fock matrix amplify into a

completely different eigenvector basis inside the degenerate subspace. The energy is insensitive to that; the theta gradient differed by eight percent between the two contraction orders. The stepwise form is therefore the implementation, not an optimization.

Canonicalization is differentiable and state-aware. The NEVPT2 denominators are defined on the canonical (natural-orbital, sub-Fock-diagonalized) set, so the contraction runs on canonicalized orbitals produced inside the differentiated path. The state-averaged variant rotates with the average density of the state-averaged solution; at one root it reproduces the ground-state canonicalization bit for bit, which is how the two are pinned against each other.

Per-state energies are read out of the eigensolver's spectrum. The state-averaged solver returns the mean of the lowest roots; the per-state energies are recovered from the running means by the sliding-mean inversion $E_k = k\,M_k - (k-1)\,M_{k-1}$. The number of roots is silently clamped by the size of the determinant space, so the inversion is performed over the effective root count, not the requested one.

Degeneracies are handled at the decomposition level. An eigensolve with degenerate eigenvalues has no unique derivative basis; the kernels use degeneracy-safe decompositions throughout, and the choice of cluster tolerance is a protocol decision with its own limitation entry (Chapter 9).

5.4 What is verified, and against what

The chain's verification record is deliberately built on invariance and cross-implementation checks rather than on agreement with any single reference: the CASSCF gradient is validated against invariance conditions and against an independent derivative-free route; the NEVPT2 energies against an independent implementation; the state-averaged addition against the single-root path at one root. Chapter 7 lists the records with their measured deviations.

Chapter 6

Second derivatives

The package differentiates its energy chain twice, so that harmonic frequencies and curvature checks come out of the same code path that produced the gradients. This chapter records the construction and the numerical facts that forced their design.

6.1 The nuclear Hessian by implicit differentiation

Differentiating through a converged orbital optimization naively — differentiating the unrolled iteration — scales with the number of iterations and inherits every numerical accident of the solver. The production route instead treats the optimized orbital parameters as an implicit function of the geometry, $\theta^*(R)$, and assembles the second derivative from the partial derivatives of the energy at the converged point plus a response term built from the orbital Hessian.

Two implementation facts came out of the verification work. The response solve must use a pseudoinverse rather than a plain solve: the orbital Hessian at a converged point is singular in the directions the optimizer is free to ignore, and the plain solve either fails or returns noise. And the implicitly assembled Hessian must be pinned against the alternative construction: the two routes agree to 1.6×10^{-11} on the validation systems, with the implicit route costing about a third of the unrolled one.

6.2 The second-order stages never materialize the Hessian

The orbital refinement (damped Newton) and the trust-region polish need the Hessian only through its action on vectors. The earlier implementation built it explicitly with `jax.hessian`, which is forward-over-forward: every intermediate of the chain is broadcast over the square of the parameter count. At five hundred parameters that construction requested a single 402 GB allocation and died. Both stages now receive Hessian-vector products — a forward-mode derivative of the gradient — which carry one tangent per reverse pass: the refinement solves its Newton step by conjugate gradients on those products, and the trust-region stage consumes them through the optimizer's `hessp` interface. The budgets are deliberately shallow (tolerance 10^{-6}, forty iterations): an inexact Newton step feeds a damping ladder and a downhill acceptance test that correct it. The deep solver cost hours of product evaluations per molecule, and the shallow budget passes the same refinement regressions.

6.3 Degenerate pairs and the derivative basis

A symmetry-degenerate pair of orbitals has no canonical derivative basis: the eigenvectors inside the degenerate subspace are arbitrary, so a naive derivative of a decomposition returns a quantity that depends on the solver's internal ordering. For a linear molecule this is not an edge case — the π pairs are exactly degenerate, and a gradient assembled through an unguarded decomposition

returns NaN. The kernels use degeneracy-safe eigensolvers throughout the differentiated path, and the second-order routes avoid differentiating decompositions where the response construction allows it.

6.4 From the Hessian to frequencies, and one cautionary record

Harmonic frequencies follow from the mass-weighted Hessian by the standard normal-mode analysis, using PySCF's driver on a Hessian this package assembles. The conversion has a layout trap that cost this project a wrong number: the Hessian must be handed over in the (atom, coordinate) block layout the thermodynamics module expects, not as a plain reshape. The wrong layout comes out plausible and silent — for CO at 6-31G it produced 1501.8 cm^{-1} against the correct 2293.5 cm^{-1}.

The frequency route is anchored against an independent value: the CO molecule at 6-31G gives 2294.2 and 2293.5 cm^{-1} from the two assembly routes, against an external anchor at 2293.5 cm^{-1}. When a validation target sits near zero, do not reach for finite differences: this project measured finite-difference errors of over one hundredfold on a near-zero mixed derivative, growing as the step shrank. Use an invariance or an analytic cross-check instead.

Chapter 7

Verification records

Every claim this package makes about itself rests on a recorded comparison. This chapter lists the records by the question they answer, with the measured deviation and the test or script that holds it. Where a check is an equality that must hold by construction, the deviation is quoted at machine precision; where it is a comparison against an independent implementation or an external reference, the deviation is quoted with its protocol.

7.1 Energy chain

Question	Measured	Where
CASCI/CASSCF against PySCF, same geometry, basis and window	10^{-12} to 10^{-10} Ha (LiH, HF, NH_3)	`t305`; driver kernels
NEVPT2 correction against an independent implementation	4×10^{-7} on the large-core test after the block-loop fix	the frozen-contraction tests
State-averaged solution at one root against the ground-state path	bit-for-bit (10^{-12})	`tests/driver/test_sa_pipeline.py`
Joint and theta-only factories at fixed geometry	10^{-10} Ha	the same suite

7.2 First derivatives

The nuclear gradient is validated by invariance rather than by agreement with a single reference, and then by a regression against an independent derivative-free route:

- the CASSCF gradient is invariant under coordinate transformations at the 10^{-14} level, and reproduces the finite-difference reference to its own noise on smooth systems;
- the regression against the analytic reference reproduces the reference to the level at which both are converged; departures appear at points where the reference geometry was not stationary, which is a property of the comparison, not of the gradient;
- a near-zero mixed derivative is a known trap for finite differences: measured errors of over one hundredfold on a derivative whose value sits near zero, growing as the difference step shrinks. Such checks use invariances instead.

7.3 Second derivatives

Question	Measured	Where
Implicit route against the unrolled construction	1.6×10^{-11}	`orbital_response_implicit`; the implicit-Hessian tests
Harmonic frequency against an external anchor (CO, 6-31G)	2294.2 and 2293.5 cm^{-1} from the two routes, against 2293.5 cm^{-1}	frequency tests
Hessian-vector product against the explicit Hessian	10^{-10} on a non-quadratic fixture	`tests/driver/test_orbitals.py`
Damped-Newton step solver (CG) against a dense solve	10^{-8} on SPD fixtures	`tests/core/test_stationary.py`

7.4 External-data cross-checks

Two independent data lines were used to check layers that have no internal ground truth:

- **A published selection-protocol data set** (DVS-APC, Zenodo record, CC-BY-4.0): the energy layer reproduces three reference state energies to 10^{-11} Ha; a port of the published entropy-based selection formula, evaluated on all 116 orbitals of every record, agrees with the published values to exactly zero difference; the singly-occupied rule of the protocol is confirmed on the full set rather than on a sample.
- **An external benchmark database** (QUEST): the state-averaged excitation pipeline is checked over fourteen states of seven molecules; the records and the family split are in Chapter 8 and the Tutorial.

7.5 The layer contracts

The six-layer architecture is itself under test: `tests/test_layer_contracts.py` (or the `lint-imports` entry point) parses the import graph and fails on any upward import. The check is structural, not conventional: an import performed lazily inside a function body is still reported, because the contract is parsed from the syntax tree.

Chapter 8

Benchmarks

Two benchmark lines exercise the package end to end. The first is a ground-state line against direct PySCF runs at matched settings; the second is the excited-state line against the QUEST reference set, whose protocol and reading are walked through in the Tutorial (Chapter on the benchmark walkthrough). This chapter records both with their conventions, so that a quoted number can be traced back to its run.

8.1 Ground-state line (M5-A)

Three closed-shell molecules at matched geometry, basis, and active window, chain against PySCF:

System	Deviation (Ha)	Note
LiH	-3.5×10^{-12}	active window [4,5], 6-31G
HF	-3.6×10^{-13}	active window [1,2], 6-31G
NH_3	-1.9×10^{-10}	active window [1,2], 6-31G

The agreement is at the level where the two codes' convergence tolerances meet. The chain's end-to-end entry reports the energy and the stationarity diagnostics; nuclear gradients on this line come from the lower-level interface and are covered by the first-derivative records of Chapter 7.

8.2 Excited-state line (QUEST subset)

Fourteen vertical excitation energies over seven molecules, 6-31G, the frontier-window protocol with its corrections, against the QUEST database's best estimates and its own SC-NEVPT2 column. The full table sits in the Tutorial; the summary by family:

Family	MAD vs best estimate	MAD vs SC-NEVPT2
$n \to \pi^*$ (8 states)	0.31 eV	0.36 eV
$\pi \to \pi^*$ (6 states)	1.14 eV	1.08 eV

Two protocol facts belong to these numbers. The basis is 6-31G while the reference set's values are near the complete-basis limit, so part of every deviation is basis, not method; and the window corrections of the Tutorial's active-space chapter move individual entries by up to a factor of two in deviation before the family statistics are computed. The residual $\pi\pi^*$ deviation — uniform in sign across five chemically different systems — is recorded as a capability boundary of the minimal-space protocol at this basis, not as a molecule-specific error.

8.3 Selection-protocol cross-check (DVS-APC)

The entropy-based active-space selection layer was checked against a published automated-selection data set on its own terms: a port of the published formula evaluated on every orbital of every record agrees with the published entropy values to exactly zero difference, the energy layer reproduces reference state energies to 10^{-11} Ha, and the protocol's singly-occupied rule is confirmed on the full data set. The cross-check also fixed a defect this package had carried: the published protocol includes a virtual-removal correction of order N that the port initially lacked; with the correction in place, the agreement statistic reaches a mean of 0.63 over the full selection set on the primary data variant and 0.59 on the alternative, roughly six times a random baseline. The per-molecule statistics, including the cases where the two variants disagree, are in the stored report.

8.4 What is not benchmarked

Large-active-space behaviour, GPU execution, spin–orbit coupled targets, and relativistic Hamiltonians have no benchmark here because they are outside the present scope (Chapter 9). A number produced by this package on those targets has no verification record behind it.

Chapter 9

Known limitations

This is the consolidated list. Each entry states the limitation, the measurement behind it where one exists, and the check a user can run. The Tutorial has the same material in shorter form; the entries here carry the numbers.

9.1 Method-family limits

Contracted NEVPT2 accuracy. The perturbation layer inherits the standard contracted-NEVPT2 approximations; the reference set's own MAE for the method is the accuracy band claimed here, not a target this package improves on.

Valence–Rydberg mixing. $\pi\pi^*$ states near Rydberg states move by up to an electron volt when diffuse functions enter the basis. Measured on ethylene at a fixed geometry: the singlet $\pi\pi^*$ vertical excitation moves from 9.27 to 8.09 eV between 6-31G and 6-31+G* in the same three-orbital window, while the triplet moves the opposite way (4.83 to 5.68 eV); the singlet–triplet gap, which the reference fixes near 3.4 eV, collapses from 4.44 to 2.41 eV. A gap collapse in the wrong direction is the signature of a mixed-in state, not a converged one. Check: re-run with and without diffuse functions.

Minimal-window limits for multiconfigurational $\pi\pi^*$ states. The benchmark record shows a uniform positive deviation of the $\pi\pi^*$ family (+0.6 to +1.5 eV) that survives every window correction tried. Check: widen the window by one occupied orbital; a deviation that shrinks is window-limited.

No spin–orbit coupling; scalar-relativistic with ECPs only. Fine structure is out of scope. **Translated functionals beyond tPBE/tPBE0.** The on-top kernel is a closed-form PBE, written out because the libxc wrapper cannot be traced; tPBE and its 0.25 hybrid tPBE0 are the covered pair. Another translated functional needs its own closed-form kernel — a one-line change in principle, a new validation in practice. Below ρ 1e-13 the kernel differs from libxc internal guards by up to 1e-4 absolute, where the grid weights put the integral contribution below 1e-25.

9.2 Implementation limits

Dense diagonalization. The CI step materializes and diagonalizes the full determinant space; the practical ceiling is a few thousand determinants per root calculation. A differentiable iterative eigensolver is future work. The (lambda, N) DVS scan inherits this ceiling: candidate windows are refused with their determinant count recorded once they exceed `max_ndet` (default 2000), which places the published (12,12)/(14,14) windows beyond the in-stack CI by design — those remain engine-side.

Frozen on-top quadrature. Geometry derivatives of the on-top energy omit the grid-velocity term by construction; the calibration bounds it at 7.3e-5 Eh/bohr on CO/6-31G at grid

level 3, falling to 2.5e-5 at level 4.

Second-derivative cost. The implicit route requires the orbital Hessian in memory and does not exploit sparsity; the unrolled route scales with the iteration count. Neither is competitive with analytic second derivatives of single-reference methods.

Memory and compile profiles of the differentiated chain. Intermediates are broadcast over the rotation-parameter count. The measured record that shaped the current design: an explicit Hessian at about five hundred parameters requested a 402 GB allocation in a single request; the Hessian-vector-product stages that replaced it carry one tangent per pass, and their first evaluation still spends minutes in XLA compilation at that size. Check: watch resident memory during the first optimization step, and leave headroom on shared machines.

9.2.1 Degenerate pairs at a stationary point

The eigensolver's cluster tolerance must separate two scales: the floating-point noise that splits symmetry-degenerate pairs at a rotated stationary point (measured between 10^{-8} and 1.2×10^{-5} on the systems in this work) and the physical gaps of the same spectrum (1.9×10^{-2} and above). A tolerance below the noise exposes the first and second derivatives to a bare inverse gap; one above the physical gaps merges distinct states. The window measured here spans 10^{-5} to 10^{-2}, and there is no admissible value when a physical gap falls inside the noise band — the fallback for that case is the decomposition-free Lanczos kernel. Check: if a linear or otherwise degenerate system returns unstable derivatives, this tolerance is the first knob to inspect.

9.3 Protocol limits

Multiple stationary points. The orbital energy is non-convex; different starting points reach different stationary points, and the selection is not stable even at fixed settings — water at 6-31G has two minima 13 kcal/mol apart, and the reference-orbital start reaches a saddle. Report the stationarity certificate and the stationary-point spread with every multireference number.

NEVPT2 convergence sensitivity. The correction depends on the orbitals more sensitively than the energy; energy-based gates are insufficient when the target is the correction. Measured: orbital stalls that move the correction by several kcal/mol across runs whose energies agree to the microhartree. Use the stability of the correction across independent runs together with a tightened orbital-gradient gate.

Window rules and bookkeeping. The frontier audit covers the patterns met so far; the electron count follows the window ($n_{\mathrm{ele}} = N_{\mathrm{ele}} - 2n_{\mathrm{core}}$) and a hard-coded count produces a wrong ground state that the energy alone does not flag. **One-step versus relaxed MC-PDFT.** The one-step orbitals are not stationary for the on-top functional (measured |grad| 3e-3 at the CASSCF point); a relaxed minimization lies 3.0e-4 Eh below on H2/6-31G. The two numbers answer different questions — papers using the one-step protocol must not be compared with relaxed values without saying which is which.

On-top columns amplify orbital-point differences. The per-state on-top energy reacts to the state-averaged orbital point at first order, where the CASSCF column reacts at second order (slope 3e-3 per radian at a CASSCF point). Two runs whose state-averaged energies agree to 1e-7 Eh can differ by 1e-5 eV in the on-top column; treat that as the column's reproducibility floor, not as noise.

Part III

Development

Chapter 10

Architecture and contracts

10.1 The six layers

Layer	Responsibility
`core`	the numerical kernel: rotations, CASCI, CASSCF, NEVPT2, stationary points. Frozen: corrections go through the verification records.
`analysis`	post-processing of converged objects; NumPy only; no kernel import.
`policy`	selection rules over analysis output.
`driver`	composition: the differentiable SCF–CASSCF–NEVPT2 pipelines, orbital-point convergence, frozen-geometry factories.
`report`	result objects and rendering.
`cli`	the command-line entry point.

Dependencies point downward only. A user script may import any layer; the layers themselves may not import upward. The most consequential of these rules is the analysis layer's isolation: it must run with nothing but NumPy installed, which is what keeps its diagnostics usable inside any environment.

10.2 Contracts are tests

The import direction is enforced mechanically, not by review: `tests/test_layer_contracts.py` parses the syntax tree of every module and fails on any upward import. The check has a deliberate property — an import hidden inside a function body is still reported, because the contract is structural. The `lint-imports` configuration in the package metadata encodes the same rules for environments where the import-linter tool is available.

10.3 How the test suite is organized

The suite mirrors the layers: `tests/core/`, `tests/analysis/`, `tests/policy/`, `tests/driver/`, `tests/report/`, plus cross-layer integration tests. Two conventions are worth knowing before adding tests:

- The analysis tests run on synthetic fixtures with hand-built numbers and no chemistry package import; this is what keeps the layer's tests runnable anywhere. The conventions under test (determinant bitmaps, AO labels) are stated in the fixtures themselves.
- The driver tests bind the layer against the kernel: the frozen factories are pinned against the joint ones, the state-averaged solution against the single-root path, and the Hessian-vector product against the explicit Hessian. When a change touches the chain, one of these bindings is the test that should notice.

10.4 Development workflow

Non-trivial additions start as a written plan, are implemented test-first, and land with the verification record that justifies them. Bug fixes follow the same rule: the record is the fix's justification, and the test is the record made executable. The differentiable-chain and second-derivative chapters were developed in that order, and several of their numerical facts are traceable to the record that found them.

Chapter 11

Reproducing the paper

11.1 Environment

Every reproduction script sets its own runtime environment at the top: the five BLAS thread variables and `JAX_PLATFORMS=cpu` before the first JAX import, and `jax_enable_x64` immediately after. A script run outside this convention may produce different timings and, in the single-precision case, wrong numbers; the settings are quoted in Chapter 2.

The scripts write JSON results next to themselves (or into a `results/` directory); the figure sources read those JSON files. Run them from a directory with the package importable, or adjust the path insert at the top of the script.

11.2 The paper's figures and tables

Script	Result
`t157_grad_gold_fix.py`	nuclear-gradient validation; invariance conditions
`t155_grad_regression.py`	gradient regression against an analytic reference
`t150_gold_refine.py`	golden-standard Hessian by re-optimizing at each displaced geometry
`t149_mix_AD_vs_FD.py`	why finite differences fail for the mixed derivative
`t151_unrolled_theta.py`	response term by unrolled iteration
`t152_generalize.py`	unrolled route on CO and N_2
`orbital_response_implicit_` `implicit_hessian.py`	response term by implicit differentiation
`t153_hessian_to_freq.py`	harmonic frequencies from the mass-weighted Hessian
`t159_basis_active_space.py`	frequency dependence on basis set and active space
`t160_collect_freq.py`	frequency summary
`t142_layerA_v2.py`, `europium_scf_convergence`, `europium_dissociation_rerun`	lanthanide dissociation energies

11.3 The later result lines

The state-averaged and benchmark lines that postdate the paper's first draft live under the project's `code/` directory:

Script	Result
`t315 / t317 / t321`	the QUEST-subset excitation runs (with `t320` the window audit)
`t316`	the assembled benchmark table (reads the recorded JSON)
`t318`	the large-molecule single-stage protocol
`t306`	the first state-averaged targets, with the two-pass weights
`t300`–`t302`	the external selection-protocol cross-check

Each of these carries its conventions in its docstring: basis, geometry source, window rule, state-weighting pass. A number quoted from one of them without those four facts is a number that will not survive being compared with the others.

11.4 Building this documentation

The Tutorial and the Manual are xelatex documents under `manual/`; compile each twice:

```
cd manual/tutorial && xelatex tutorial.tex && xelatex tutorial.tex
cd ../manual    && xelatex manual.tex    && xelatex manual.tex
```

Part IV

Acceleration and performance

Chapter 12

Resolution of identity (RI/DF)

The four-center two-electron integrals of the multireference chain are the memory bottleneck of its nuclear-derivative path. Under `pyscfad`, the derivative of `int2e` allocates a dense `nctr` $\times$ `nao`^{4} array before any chemistry happens: 55.0 GB at `nao` = 96 and 2033 GB at `nao` = 175 for the systems measured here. The module `diffmrc.core.ri` replaces the four-center tensor in the correlation stage by two three-center factors and a metric, and supplies a nuclear-derivative path whose working set is `nao`^{2} $\times$ `naux` per atom. This chapter states the approximation, the public interface, the axis and sign conventions, the measured error budget, the memory figures, and the boundaries of the implementation.

12.1 The approximation

The RI (density-fitting) approximation writes

$$(pq|rs) \;\approx\; \sum_{PQ} (pq|P)\, S^{-1}_{PQ}\, (Q|rs), \qquad S_{PQ} = (P|Q), \tag{12.1}$$

where capital indices run over an auxiliary basis and S is the auxiliary metric. With the three-center tensor $B_{\mu\nu P} = (\mu\nu|P)$ and the symmetric inverse square root $J = S^{-1/2}$, the four-center tensor is assembled as

$$W = BJ, \qquad (\mu\nu|\lambda\sigma) = \sum_{P} W_{\mu\nu P} W_{\lambda\sigma P}, \tag{12.2}$$

that is, one matrix product followed by one GEMM over the pair index. The two-step form is the implementation, not an optimization: an algebraically equivalent single four-operand einsum showed absolute errors of order 10^{-6} on a 96^4 array on this stack, and two such chains differed by eight percent in the rotation gradient.

The inverse square root is computed from the symmetric eigendecomposition of S. Directions with eigenvalue below `rcond` $\times$ the largest eigenvalue are truncated to zero in J; the default `rcond` is 10^{-12}, and the number of dropped directions is reported by the factor container.

The derivative path uses the factor-level form of Eq. (12.2): the host code supplies B, J and their nuclear derivatives, and a pure-JAX function with an explicit `custom_jvp` maps (B, J) to the four-center tensor. No four-center derivative integral is ever formed, and no eigendecomposition is differentiated inside JAX; the derivative of J is the analytic Daleckii–Krein divided difference of $x^{-1/2}$, which is finite at degenerate metric eigenvalues.

12.2 API reference

The functions below are exported by `core.ri`. Shapes follow the conventions of Section 12.3.

Entry point	Returns and purpose
`make_auxmol(mol, auxbasis=None)`	`(auxmol, label)`. Tries the names in `AUX_BASIS_PRIORITY` and falls back to the orbital basis; a basis that produces zero functions counts as a failure.
`ri_three_center(mol, auxmol)`	B, shape `(nao, nao, naux)`; uses `df.incore.aux_e2`, not `mol.intor('int3c2e_sph')`.
`ri_metric(auxmol)`	S, shape `(naux, naux)`, symmetrized.
`inv_sqrt_metric(S, rcond, eig=None)`	Truncated symmetric $S^{-1/2}$, shape `(naux, naux)`.
`ri_factors(mol, auxmol, auxbasis=None, rcond)`	`RIFactors` carrying B, S, J, the metric eigendecomposition, the basis label and `rcond`.
`ri_eri_ao(mol, auxmol, factors=None, rcond)`	RI four-center tensor, shape `(nao, nao, nao, nao)`, chemist notation.
`ri_eri_mo(C, b, jhalf=None, full=True)`	`(b_mo, eri_mo)` with shapes `(norb, norb, naux)` and `(norb, norb, norb, norb)`; `eri_mo` is `None` when `full=False`.
`ri_three_center_deriv(mol, auxmol, atom)`	dB/dR_{atom}, shape `(3, nao, nao, naux)`.
`ri_metric_deriv(auxmol, atom)`	dS/dR_{atom}, shape `(3, naux, naux)`.
`ri_factors_deriv(mol, auxmol, atom, factors)`	`RIFactorDerivs` with `db`, `ds` and `djhalf`.
`ri_eri_deriv_contract(gamma, factors, derivs, per_axis=False)`	Contracts $\gamma = \partial E/\partial(\mathrm{eri_ao})$ with one atom's derivative; scalar, or a length-3 vector when `per_axis=True`.
`ri_eri_from_factors(b, jhalf)`	Pure-JAX RI four-center tensor with the analytic `custom_jvp`.
`ri_eri_jvp(b, jhalf, db, djhalf)`	Convenience wrapper returning the value and the directional derivative.

Two further entries exist for validation only. `ri_eri_deriv` materializes the full `(3, nao, nao, nao, nao)` derivative and refuses to allocate above a caller-supplied byte limit; production code uses the factor derivatives with `ri_eri_deriv_contract` or the JAX rule instead. `inv_sqrt_metric_jvp` exposes the metric derivative on its own.

The intended nuclear-gradient recipe is:

```
from diffmrc.core import ri

factors = ri.ri_factors(mol, auxmol)              # host, numpy
E = energy_from_factors(factors.b, factors.jhalf) # pure JAX chain
dE_db, dE_djhalf = jax.grad(E, argnums=(0, 1))(
    factors.b, factors.jhalf)
for atom in range(mol.natm):
    d = ri.ri_factors_deriv(mol, auxmol, atom, factors)
    dE_dR[atom] = (np.sum(dE_db * d.db)
                   + np.sum(dE_djhalf * d.djhalf))
```

The only derivative integrals touched are the three-center `ip1`/`ip2` arrays and the metric `ip1`/`ip2` arrays.

12.3 Axis and sign conventions

The conventions are fixed in the module and verified by isolated finite differences; they are repeated here because the `libcint` names are misleading.

- $B[\mu, \nu, P] = (\mu\nu|P)$: both orbital indices first, the auxiliary index last. $S[P, Q] = (P|Q)$.
- `eri_ao`$[\mu, \nu, \lambda, \sigma] = (\mu\nu|\lambda\sigma)$ in chemist notation.
- `int3c2e_ip1` differentiates the *first* orbital center and `int3c2e_ip2` differentiates the *auxiliary* center, not the second orbital center. `int3c2e_ip3` is not exported by `libcint` on this stack.
- The `ip` series carries the electronic-coordinate sign: $\partial(\mu\nu|P)/\partial R_{c(\mu)} = -$`ip1`. The second orbital center is $-$`ip1` with the orbital-pair axes exchanged, and the auxiliary center is $-$`ip2`.
- For the metric, $dS/dR_A = -($`ip1`$[P \in A] +$ `ip2`$[Q \in A])$.

The isolated checks displace the molecule with the auxiliary basis fixed and the auxiliary basis with the molecule fixed: the two groups of terms reproduce their finite differences to 7.9×10^{-11}, and the sum reproduces the full displacement to 4.8×10^{-11}.

12.4 Error budget

Three error classes are reported separately, because they have different origins and different remedies.

Class	Measured value and meaning
Implementation	RI assembly against the pyscf DF reconstruction: 5.8×10^{-13} (H_2O/6-31G) and 1.3×10^{-12} (N_2/cc-pVDZ). This is the numerical floor of the contraction, not a physical error.
RI physical	The approximation of Eq. (12.1) at fixed orbitals and CI vector: NEVPT2 Δe_2 of $+1.1 \times 10^{-5}$ (N_2/cc-pVDZ) and -7.2×10^{-6} Ha (CO/cc-pVDZ); -1.8×10^{-6} Ha on ThO/6-31G with the `autoaux` fitting set.
Chain derivative	RI-CASCI gradient against a finite difference of the same RI energy: 9.6×10^{-8} (H_2O/6-31G) and 3.1×10^{-7} (N_2/cc-pVDZ), absolute, in Eh/bohr.

The derivative integrals themselves agree with central differences at a 10^{-4} bohr step to 3.6×10^{-9} (three-center tensor) and 1.2×10^{-9} (metric) on H_2O/6-31G. The materialized ERI derivative shows 2.5×10^{-8} on the same system with the default fitting basis, whose metric condition number is 2.5×10^7; switching to a fitting basis with condition number 2.5×10^5 brings every component below 6.7×10^{-9}. The residual on the default basis is the finite-difference floor amplified by the metric condition number, not an implementation defect.

The physical error is controlled by the fitting basis, not by the code. The values above sit at the lower edge of the usual RI band (10^{-5}–10^{-3} Ha for correlation energies) and must be quoted with the fitting basis and the system attached.

12.5 Memory

The value path still materializes the dense `nao`^{4} tensor, because that is the object the CI layer consumes. The derivative path does not: its per-atom working set is three three-index arrays of size `nao`$^2 \times$ `naux`, plus the metric derivative.

nao	naux	RI chain peak RSS	dense derivative array	array ratio
96	646	2.11 GB	55.0 GB	26×
175	746	16.05 GB	2033 GB	127×

The peaks cover the integral-and-derivative chain only, without SCF or CI. The `nao` = 96 row uses the orbital-basis fallback and the `nao` = 175 row the `autoaux` route; the memory scaling depends on `naux`, not on which fitting route produced it. The derivative-array ratio in the table is 26× at `nao` = 96 and 127× at `nao` = 175. Against the measured dense whole-chain peak of 210–213 GB at `nao` = 96, the end-to-end reduction is about 100×. The per-atom derivative factors occupy 152.9 MB at `nao` = 96.

12.6 Boundaries

- *The SCF layer remains dense.* RI removes the memory wall of the correlation stage, not of the reference determinant. A full-chain nuclear gradient for an actinide still requires an RI-JK SCF or an explicit frozen-reference partial-derivative semantics.
- *Second derivatives are not implemented.* The factor-level `custom_jvp` is first order. Differentiating the reverse pass a second time is possible in principle but is outside the validated surface; the analytic $S^{-1/2}$ rule is available first-order only.
- *Actinide fitting sets are missing.* The def2 and cc-pVDZ-RI fitting families stop at Rn. For Th the module falls back to the orbital basis (`naux` = 646, metric condition number 1.5×10^9), which shifts a CASCI energy by −0.594 mHa. The even-tempered construction `aug_etb_for_dfbasis` is the best available substitute (+0.021 mHa, `naux` = 625); a dedicated actinide JKfit basis remains an open item.
- *Basis-family matching is the caller's responsibility.* `make_auxmol` implements a priority list, not a family-aware selector. A cc-pVDZ orbital basis should be paired with cc-pVDZ-RI explicitly.
- *The truncated metric has a floor.* `rcond` defaults to 10^{-12} and no direction was dropped in the systems measured here, but the Th fallback metric is already at the 10^9 condition number where the truncation choice starts to matter.

12.7 A standalone example

`examples/ri_df.py` in the repository runs the H_2O/6-31G CAS(2,2) case end to end: assembly against the pyscf DF solve, RI-CASCI, the three-center and ERI derivatives against central differences, and the JAX rule. The core of the script is:

```
import numpy as np
from pyscf import gto, scf
from diffmrc.core import ri
from diffmrc.core.casci import cs_casci

mol = gto.M(atom='O 0 0 0; H 0.7557 0 0.5880; H -0.7557 0 0.5880',
            basis='6-31g', unit='Angstrom', verbose=0)
mf = scf.RHF(mol).run(verbose=0)
auxmol, label = ri.make_auxmol(mol)
factors = ri.ri_factors(mol, auxmol)

eri_ri = ri.ri_eri_ao(mol, auxmol, factors=factors)
eri_dense = mol.intor('int2e', aosym='s1')
```

```
c = mf.mo_coeff
h1_mo, eri_mo_dense = mo_integrals(c, mf.get_hcore(), eri_dense)
b_mo, eri_mo_ri = ri.ri_eri_mo(c, factors.b, factors.jhalf)
e_ri, ci_ri = cs_casci(h1_mo[:6, :6], eri_mo_ri[:6, :6, :6, :6],
                       ncore=4, nact=2, nele_act=2, return_ci=True)

for atom in range(mol.natm):
    d3 = ri.ri_three_center_deriv(mol, auxmol, atom)
    # compare d3 against a central difference of ri_three_center
```

The measured run on the H_2O/6-31G system prints an assembly identity residual of 1.1×10^{-12}, a dense-vs-RI RMS of 8.3×10^{-6}, an RI-CASCI shift of $+1.05 \times 10^{-5}$ Eh, a worst three-center derivative deviation of 3.6×10^{-9}, and a worst ERI derivative deviation of 3.7×10^{-8}, in 6.4 s of script time. The companion `examples/ri_nevpt2.py` reports the RI-NEVPT2 correction, $+1.7 \times 10^{-5}$ Ha from the dense value on the same system.

Chapter 13

Performance and numerical hygiene

This chapter records the implementation choices that carry a performance or numerical-branch consequence: the ordering of the MO integral transform, the blocked CI solver, the automatic solver selection, the jit reference-array form, and one repeated contraction in the CI matrix–vector product. Each choice is an engineering decision with a measured anchor; none changes the definition of the energy. The measurements were taken on one 64-core node with JAX in double precision on CPU, with the BLAS thread count fixed at one; wall clock is quoted as a relative quantity only, while compilation counts and peak resident memory are load-independent.

13.1 MO integral transforms: four sequential contractions

The two-electron transform is the most expensive single contraction in the differentiable chain, and its algebraic ordering is part of the numerical contract. The implementation is `core.nevpt2.impl.stepwise_mo_ints`, which performs four sequential contractions,

$$h_{ij} = \sum_{ab} C_{ai}\, h_{ab}\, C_{bj}, \qquad (ij|kl) = \sum_{pqrs} C_{pi} C_{qj} C_{rk} C_{sl}\, (pq|rs), \tag{13.1}$$

as one two-index transform followed by three one-index transforms:

```
def stepwise_mo_ints(Cx, h_, e_):
    h1 = jnp.einsum('ai,ab,bj->ij', Cx, h_, Cx)
    z = jnp.einsum('pi,pqrs->iqrs', Cx, e_)
    z = jnp.einsum('qj,iqrs->ijrs', Cx, z)
    z = jnp.einsum('rk,ijrs->ijks', Cx, z)
    eri = jnp.einsum('sl,ijks->ijkl', Cx, z)
    return h1, eri
```

Every MO transform in the differentiable chain calls this function, including the three canonicalization transforms and the state-averaged driver. A second copy in the driver delegates to it, so the two call paths cannot drift apart.

13.1.1 Why the one-shot form is not used

The one-shot form contracts four orbital coefficients against the four-index integral in a single expression, `jnp.einsum('pi,qj,rk,sl,pqrs->ijkl', C, C, C, C, eri)`. It is algebraically identical and was the original implementation. Two measurements removed it.

First, on a 96^4 integral array the one-shot form carried an absolute error around 10^{-6} against the sequential transform (the recorded value is 1.28×10^{-6}, with the active block agreeing to 8.9×10^{-16}). The error is harmless for the energy, which is insensitive to it, but a linear molecule's generalized Fock matrix contains exact π and δ degeneracies whose gaps are at the 10^{-15} level. A perturbation of 10^{-6} against a gap of 10^{-15} rotates the eigenvector basis inside the degenerate

subspace completely; the ratio is about 10^9. The theta gradient differed by eight percent between the two contraction orders, and the Hessian-vector product was NaN for the one-shot form and finite for the stepwise one.

Second, the sequential form is faster at production size, not slower. On the physical ThO/ECP array with 96 orbitals ($\max|eri| = 4.78$), the one-shot evaluation took 5.08–5.79 s and the stepwise evaluation 0.417–0.467 s, a factor of eleven to thirteen; on a random array of the same shape both forms finished in about 50 μs. The difference is the contraction path XLA selects for the physical memory layout, and it is a runtime effect, not a reproducibility precaution.

A release-tree cross-check qualifies the first measurement. On the same physical array the current one-shot form agrees with the stepwise form to 3.33×10^{-16} (rms 5.73×10^{-18}), so the 10^{-6} defect was not reproduced in this JAX version and on this array. The ordering is retained for both reasons above: the branch sensitivity of a degenerate subspace is a property of the contraction order rather than a fixed numerical offset, and the sequential form is the faster route at the size where the transform dominates.

13.1.2 Accuracy on the release tree

The two orders were compared at fixed geometry and fixed theta, before and after the substitution. The differences are at or below 5.7×10^{-14} Ha for the energy and 2.7×10^{-14} for the gradient; the non-degenerate H2O case is bit-identical.

System and quantity	ΔE (Ha)	$\Delta g_{\max}$
H2O/6-31G CAS(2,2), CASSCF energy and theta gradient	0	0
CO/6-31G CAS(4,4), linear, CASSCF energy and theta gradient	5.7×10^{-14}	2.7×10^{-14}
H2O/6-31G CAS(2,2), NEVPT2 energy (eager and jit)	0	—
H2O/6-31G CAS(2,2), NEVPT2 theta gradient	—	6.7×10^{-15}

The acceptance band for the substitution was 10^{-12} Ha on non-degenerate systems; the largest observed energy difference is 5.7×10^{-14}.

13.2 The S_z-blocked CI solver

`core.casci.cs_casci_blocked` diagonalizes the CI Hamiltonian one spin-projection block at a time. The partition is exact: the Hamiltonian is strictly zero between different S_z blocks (the largest measured off-block element is 0.000×10^0), so the global lowest eigenvalue is the lowest eigenvalue over the block minima. For H2O/6-31G CAS(2,2) the block structure is one determinant at $S_z = +6$, four at $S_z = +8$, and one at $S_z = +10$; the global minimum lies in the four-determinant block, whose smallest in-block gap is 2.545×10^{-2}, and the block energies are separated by 0.34 Ha.

The solver exists because the unblocked CI matrix carries a three-fold degeneracy at -84.8416418 Ha for H2O/6-31G CAS(2,2). The reverse rule of a dense `eigh` forms $1/(w_j - w_i)$ for every pair in the matrix and returns NaN at that degeneracy; inverse iteration defers the NaN from the first derivative to the second but does not remove it. Block separation is necessary but not sufficient: the reverse pass forms the same factor for every pair inside a block whether or not the pair carries cotangent, so one bit-exact pair anywhere in any block turns the untouched entries into $0 \times \infty$. The block eigenproblem therefore goes through `core.eigh.eigh_safe`, whose softened rule freezes the rotation inside a degenerate subspace.

13.2.1 Memory and time at two determinant counts

The two anchor spaces are CAS(6,6) with 924 determinants and CAS(8,8) with 12870 determinants, both on H2O/6-31G. The 12870 case used ncore = 1. Peak resident memory was measured under a cgroup limit; the plain 12870 run required a 48 GB limit and reached 37.83 GB.

Route	E (Ha)	Time	Peak RSS
plain, eager	−85.21730226017026	27.65 s	37.83 GB
blocked, eager	−85.21730226017024	84.1–88.7 s	6.78 GB
blocked, jit	same	41.70 s (1.97 compile + 39.73 evaluate)	7.78 GB
plain, jit	—	not run	—

The memory ratio is 5.58 in eager execution and 4.86 in the jitted form. The time direction is the opposite: blocked eager is 3.2 times slower than plain eager, and the blocked jit evaluation (39.73 s, excluding its 1.97 s compile) is still 1.44 times the plain eager time, although it is 2.1 times faster than its own eager form. The plain jitted form was not run because the same unblocked CI graph already exceeded 2400 s of compilation at 924 determinants; the 12870-determinant graph is larger. The blocked solver's benefit is therefore memory and compilation feasibility, not single-call time: it builds one block at a time, so the largest boolean tensor drops from $(12870, 12870, 18)$ to $(4900, 4900, 18)$ at the cost of per-block dispatch. The energy difference between the two routes is 2×10^{-14} Ha.

At 924 determinants the picture reverses: plain takes 14.7–16.9 s at 1.20–1.23 GB and blocked takes 27.8–29.8 s at 1.26–1.33 GB. There is no memory benefit and the eager solver is 1.8 times slower.

13.3 Automatic solver selection

`core.nevpt2.impl.resolve_ci_solver` maps `ci_solver='auto'` to one of the two solvers from the determinant count,

$$N_{\mathrm{det}} = \binom{2\,n_{\mathrm{act}}}{n_{\mathrm{ele,act}}}, \tag{13.2}$$

with the threshold `BLOCKED_CI_MIN_DET` = 2000: at or above it the blocked solver is selected, below it the plain solver. An explicit `'plain'` or `'blocked'` always wins, and any other value raises `ValueError`. The threshold is an engineering choice between the 924 and 12870 anchors, not a measured optimum; the crossover between the two anchors was not measured.

```
from diffmrc.core.nevpt2.impl import (canonicalize_jax,
                                      resolve_ci_solver)

# Inspect the decision without running a calculation:
solver = resolve_ci_solver('auto', nact, nele_act)   # 924 -> 'plain'

# canonicalize_jax accepts 'auto', 'plain', or 'blocked':
C_can, ci_can, eps = canonicalize_jax(
    C, h1e, eri, ci, ncore=ncore, nact=nact, nele_act=nele_act,
    ci_solver='auto')
```

The flag applies to the two internal CI re-solves of the canonicalization routines (`canonicalize_jax` and `canonicalize_sa_jax`). The main CI solve of the production chain remains on the plain solver deliberately: the blocked solver returns the exact eigenvector, while the plain solver returns an inverse-iteration vector with a residual of order 10^{-8}, and substituting the vector would change the NEVPT2 correlation energy by more than the 10^{-12} acceptance band. In the state-averaged routine the flag also has no effect when more than one root is requested, because there is no multi-root blocked solver and the state-average branch of `cs_casci` is required.

13.3.1 What the switch leaves invariant

Orbitals and orbital energies are bit-identical between the two routes; the CI vector differs by a global phase and by the plain solver's residual. The values below are the phase-aligned differences.

System	N_{det}	$\max \lvert\Delta C\rvert$	$\max \lvert\Delta\epsilon\rvert$	CI difference
H2O/6-31G CAS(2,2)	6	0	0	4.71×10^{-10}
CO/6-31G CAS(4,4)	70	0	0	5.39×10^{-8}

The identity is the intended semantics: the internal re-solve enters no returned quantity except the CI vector, while the natural-orbital rotation and the sub-block Fock diagonalization are built from the input vector.

13.3.2 The small-system penalty the default avoids

Forcing the blocked solver on a small space is a net cost. On the H2O/6-31G CAS(2,2) NEVPT2 chain, the eager correction takes 23.50 s with the plain solver and 31.41 s with the blocked solver, a 36 percent increase, and the end-to-end wall clock moves from 60.3 s to 74.2 s. With `'auto'` the same chain resolves to plain and returns to 22.26 s eager and 62.8 s wall clock, within the load band of the pre-change measurement. The measured anchors behind the threshold are collected below.

N_{det}	System	plain	blocked	Decision
6	H2O CAS(2,2)	23.50 s eager, 60.3 s wall	31.41 s eager, 74.2 s wall	plain
70	CO CAS(4,4)	9.5–16.4 s	17.2–24.6 s	plain
924	H2O/CO/N_2 CAS(6,6)	14.7–16.9 s, 1.20–1.23 GB	27.8–29.8 s, 1.26–1.33 GB	plain
12870	H2O CAS(8,8)	27.65 s, 37.83 GB	84.1–88.7 s, 6.78 GB	blocked

13.4 Reference arrays as jit arguments

The theta-only factories compute the reference SCF and the AO integrals once, at construction, and return a function of the rotation parameters. The reference arrays can enter that function in one of two ways: closed over, or passed as traced arguments. In the closure form the arrays become constants of the traced graph, so a jitted wrapper compiles one executable per geometry. In the array form the arrays are runtime arguments, and one executable serves every geometry whose arrays have the same shape and dtype.

The API exposes both members on the returned callable: `E_theta.component_args` is the array-argument function and `E_theta.arrays` is the tuple to pass, in order. The state-averaged energy factory `make_sa_energy_fn` carries the same pair, and the MC-PDFT table passes them through `converge_orbital_point(..., arrays=E_sa.arrays)` so that the orbital search does not compile the AO integrals into its graph.

```
import jax

E_theta = make_nevpt2_theta_fn(mol, ncore, nact, nele, nmo, pairs,
                               scf_builder)
E_jit = jax.jit(E_theta.component_args)   # one executable
E_ref = E_jit(theta, *E_theta.arrays)     # for every same-shaped geometry

# The orbital-point search takes the same pair:
converge_orbital_point(E_theta.component_args, len(pairs),
                       arrays=E_theta.arrays)
```

The measured difference on three geometries (the reference scaled by −1 percent, 0, and +1 percent about the center of mass) is one compile against three, and 2.67 s against 8.60 s for the three calls, measured as interleaved medians.

Form	jit objects / compilations	Total for three geometries
closure constants	3 / 3	8.60 s
array arguments	1 / 1	2.67 s

The compile count is the load-independent quantity; the wall clock matters at production scale, where a single f-block window compile is of order fifteen minutes. Three closures then cost about forty-five minutes against fifteen minutes plus two evaluations in the array form.

One restriction belongs to this section. The geometry-dependent joint entry `make_nevpt2_joint_fn` cannot be traced under `jax.jit` in this pyscfad build, because the SCF it runs inside the call raises a tracer-conversion error. The array-argument form is therefore a fixed-geometry device and belongs to the theta-only factories. The correlation contraction itself is inside the jitted theta-only function: the NEVPT2 energy in eager and jitted execution agrees bit for bit, so the untraceable part is the SCF, not the second-order correction.

13.5 Repeated core folding in the CI matrix–vector product

`core.ci_matvec` builds the effective one-electron operator once per matrix–vector product. The original code recomputed that fold separately inside the apply and diagonal paths, so every product paid it twice. The apply path now receives the folded quantities $(h_{\mathrm{eff}}, g, e_{\mathrm{core}})$ from a shared helper, and the diagonal path keeps its own call. The output is unchanged bit for bit: the norm of the test vector is 3977.4197449984595 in every interleaved run before and after the change, and the six leading components agree as well.

The timing consequence is small. In eager execution the apply routine's cumulative time moved from 5.49 s to 5.36 s, about 2.5 percent and inside the load noise; after jitting, one product takes 10–30 ms and the before/after distributions are indistinguishable. This is a code-hygiene change, not a performance promise.

13.6 Choosing among the options

Option	Default	When to change it
`ci_solver`	`'auto'`	Keep the default. Force `'plain'` only when the exact pre-change CI vector must be reproduced; force `'blocked'` only on a large determinant space where the plain tensors do not fit, and expect the single-call time to grow.
MO transform	stepwise	Always use `stepwise_mo_ints`; the one-shot form is not exposed as an option.
jit reference arrays	array form	Use `component_args` and `arrays` for every repeated geometry. Do not jit the joint geometry-dependent entry.
`ci_matvec` folding	single fold	No user action; the change is internal and numerically neutral.

Two properties of the measurements above belong to their reading. Wall-clock numbers are relative and were taken on one shared node with the load recorded; the compile counts, the peak resident memory, and the energy and gradient differences are the reproducible quantities. The 2000-determinant threshold is a choice between two measured anchors, and a user whose determinant count lies between them should measure the two routes on that space rather than assume the default.

Part V

New in version 0.3

Chapter 14

New modules in version 0.3

Version 0.3 adds five layers to the public surface: a relaxed-path NEVPT2 nuclear gradient, a coupled CASPT2 contraction, a valence-level spin-orbit mean field for effective-core potentials, a RIJONX self-consistent-field driver, and a scalar density-fitted X2C CASSCF layer. Each is additive: no existing signature changed, and the release core described in Chapter 5 is untouched. This chapter states, for each layer, what the entry point computes, what the caller must supply, and where the implementation stops. The RI factor conventions reused by two of the layers are those of Chapter 12.

Module	Main entry point	Layer
`driver.nevpt2_zvector`	`nevpt2_relaxed_gradient_zvector`	response on top of the differentiated chain
`core.caspt2`	`caspt2_contracted_e2_coupled`	value (NumPy)
`core.somf_ecp`	`valence_somf_matrix`	value (NumPy)
`driver.ri_scf`, `driver.ri_scf_driver`	`rijonx_scf`, `rijonx_scf_gradient`	SCF (pyscfad)
`core.x2c_casscf_df`	`df_x2c_casscf`	value and analytic gradient

14.1 Relaxed NEVPT2 nuclear gradients

`driver.nevpt2.nevpt2_nuclear_gradient` differentiates the frozen-theta energy: the SCF reference and the CI vector are rebuilt at every displaced geometry, but the orbital-rotation parameters θ stay at their stationary values. Along the physical path θ follows the CASSCF stationarity condition, and NEVPT2 is not variational in the orbitals, so the frozen partial is not the relaxed derivative. The correction is the Z-vector response

$$g_{\mathrm{relaxed}} = g_{\mathrm{frozen}} - M^T H_{\theta\theta}^{-1} s, \qquad H_{\theta\theta} = \frac{\partial^2 E_{\mathrm{CASSCF}}}{\partial \theta^2}, \quad M = \frac{\partial^2 E_{\mathrm{CASSCF}}}{\partial \theta \, \partial R}, \quad s = \frac{\partial e_2}{\partial \theta}, \tag{14.1}$$

with $e_2 = E_{\mathrm{NEVPT2}} - E_{\mathrm{CASSCF}}$. The vector $z = H_{\theta\theta}^{-1} s$ is the orbital Z-vector: one linear solve per state serves every Cartesian direction. The CI coordinates need no separate multiplier in this reduced space, because both energy functions solve the CI eigenvalue problem internally; the CI block of the full Hessian is replaced by its Schur complement, which is $H_{\theta\theta}$ as written.

Entry point	Returns and purpose
`make_nevpt2_joint_fn_si_exact(...)`	$E(\mathrm{mol}, \theta)$ with the exact near-zero `Si` channels; same contract as `driver.nevpt2.make_nevpt2_joint_fn`. Takes the space definition, the orbital pairs, the SCF builder, `e2_scale`, and optionally `align_ref` and `align_bounds`.
`align_reference_blocks(...)`	Differentiable blockwise alignment of the SCF coefficients onto a fixed reference; arguments `(C, S, C_ref, bounds)`, where `bounds` is a list of `(lo, hi)` orbital ranges.
`orbital_zvector(...)`	Solves $H_{\theta\theta} z = s$ by truncated pseudo-inverse; arguments `(H_tt, source, rcond=None, symmetrize=True)`; returns `(z, info)` with the spectrum, condition number and residual.
`lagrangian_relaxed_gradient(...)`	The contraction $g_{\mathrm{frozen}} - M^T z$; arguments `(g_frozen, z, M)`.
`nevpt2_relaxed_gradient_zvector(...)`	The assembled relaxed gradient; returns `(gradient, info)`. Required arguments: `mol`, `theta`, `E_casscf`, `E_nevpt2`; optional `rcond`, `H_tt`, `M`, `source`, `g_frozen`.
`si_energy_from_invariants(...)`	The `Si` channel sum over `(norm, ener, diff)`; `mode='masked'` reproduces the release value mask for diagnostics only.

Three settings carry the accuracy of the response.

- *Reference convergence.* The reference SCF is converged with `conv_tol` of 10^{-13} and `conv_tol_grad` of 10^{-10} at every geometry. The response operator is a second derivative of the same chain, so its accuracy is bounded by the orbital-gradient convergence of the reference, not by the energy convergence alone.
- *Blockwise alignment.* The SCF solution is defined only up to orthogonal mixing inside the core, active and virtual blocks, and two neighbouring geometries can return different representatives (a phase flip, or an ordering change inside a near-degenerate pair). Because θ is defined relative to that representative, an unaligned reference makes the same θ mean different orbitals: on H_2O/6-31G the measured fixed-theta energy slope was -5.8 Eh/bohr against -0.0528 Eh/bohr in the aligned gauge. Pass the base-geometry coefficients as `align_ref` and the block bounds as `align_bounds`; the map is the identity at the base geometry.
- *Exact near-zero `Si` channels.* The release core evaluates the `Si` class as $-\sum_i n_i/(d_i + h_i/n_i)$ with a value mask $|n_i| > 10^{-12}$, where n_i is the channel norm, d_i its zeroth-order denominator and h_i the core-coupling term. Symmetry can drive an individual n_i to 10^{-13}: dropping such a channel changes the energy by about 10^{-13} Eh but its derivative by order 0.1–1, because the mask is a discontinuity in the energy, not a removable singularity. The module evaluates the same algebra as $-\sum_i n_i^2/(d_i n_i + h_i)$ with the guard moved to the denominator. Only the `Si` class is treated this way; the other four contraction classes keep the release masks.

The validation covers closed-shell, frozen-core H_2O/6-31G and LiH/6-31G, CAS(2,2), against finite differences of the relaxed path. Open-shell, ECP, f-block and large-basis systems are outside the validated range. A minimal call is

```
import jax.numpy as jnp
from pyscfad import scf

from diffmrc.core.orblib import orbital_pairs
from diffmrc.driver.nevpt2_zvector import (
    make_nevpt2_joint_fn_si_exact, nevpt2_relaxed_gradient_zvector)
```

```
ncore, nact, nele = 1, 2, 2
nmo = mol.nao_nr()
pairs = orbital_pairs(ncore, nact, nmo, include_cv=True)

def builder(m):
    mf = scf.RHF(m)
    mf.conv_tol = 1e-13
    mf.conv_tol_grad = 1e-10
    return mf

mf = builder(mol)
mf.kernel()
bounds = [(0, ncore), (ncore, ncore + nact), (ncore + nact, nmo)]
ref = jnp.asarray(mf.mo_coeff)

# theta: CASSCF stationary parameters at this geometry
E_casscf = make_nevpt2_joint_fn_si_exact(
    ncore, nact, nele, nmo, pairs, builder, e2_scale=0.0,
    align_ref=ref, align_bounds=bounds)
E_nevpt2 = make_nevpt2_joint_fn_si_exact(
    ncore, nact, nele, nmo, pairs, builder, e2_scale=1.0,
    align_ref=ref, align_bounds=bounds)

grad, info = nevpt2_relaxed_gradient_zvector(
    mol, theta, E_casscf, E_nevpt2, rcond=1e-7)
```

The cutoff `rcond` is a relative cutoff on the pseudo-inverse. The H_2O CASSCF Hessian carries one soft mode (eigenvalue 3.8×10^{-6}, condition number 2.3×10^{7}) whose response is not part of the correlation source; 10^{-7} cuts it, while a looser cutoff keeps it and amplifies roundoff.

14.2 Coupled CASPT2

`core.caspt2` has two contraction schemes of the same eight-class internally contracted basis. The diagonal path (`caspt2_contracted_e2`) evaluates each contracted function with its own denominator,

$$e_2 = -\frac{W^2}{A - E_0 S}, \tag{14.2}$$

which is the CASPT2-D approximation. The coupled path replaces the diagonal zeroth-order Hamiltonian by the state-specific generalized Fock operator $H_0 = \sum_{pq} f_{pq} E_{pq}$. Its off-diagonal blocks (inactive–active, active–virtual, inactive–virtual) connect determinants of different excitation classes, and the amplitudes solve the projected linear system

$$(A - E_0 S)\, x = -W, \qquad A_{pq} = \langle K_p | H_0 | K_q \rangle, \quad S_{pq} = \langle K_p | K_q \rangle, \quad W_p = \langle K_p | H | 0 \rangle, \tag{14.3}$$

with $e_2 = \sum_p W_p x_p$ and the class decomposition $e_{2,\mathrm{class}} = \sum_{p \in \mathrm{class}} W_p x_p$. The overlap matrix S is regularly linearly dependent; it is diagonalized and eigenvectors with eigenvalue below `rtol` $\times$ $\max(1, \lambda_{\max})$ are dropped.

Entry point	Returns and purpose
`caspt2_contracted_e2_coupled(...)`	`(e2_total, breakdown)`, plus the assembled `S`, `A`, `W`, `E0`, `x` and the class labels when `return_system=True`. Arguments: the MO integrals, the CI vector and determinant bitmaps, `ncore`, `nact`, `nele`; optional `fock` and `rtol`. With `fock` omitted, the state-specific generalized Fock is built from the CI vector and the integrals.
`caspt2_contracted_e2`, `caspt2_contracted_e2_jax`	The diagonal-Fock path and its differentiable twin; the latter is the jax chain that can be traced in the CI coefficients.

The relative weight of the `VM` class follows the reference implementation: with the two orderings written cross and direct, the Molcas normalisation is $V_P = \text{cross} + \text{direct}$ and $V_M = \sqrt{3}\,(\text{cross} - \text{direct})$. The factor $\sqrt{3}$ multiplies the whole `VM` function and cancels in $W^2/(A - E_0 S)$, so the operator content carried here is cross − direct, that is, a relative weight of -1 between the two orderings. Both components enter the energy independently for off-diagonal pairs and coincide for diagonal pairs. Version 0.3 corrects the previous convention for this component.

The coupled total against the reference CASPT2 class table is 1.0081 on H_2O/6-31G and 1.0100 on CO/6-31G, both CAS(2,2). The path is a NumPy value layer: it has no jax twin, and the differentiable CASPT2 route remains the diagonal-Fock contraction `caspt2_contracted_e2_jax` at the CASPT2-D level. The cost is n_{fn}^2 function-pair contractions plus one H_0 application per function, intended for small and medium active spaces.

```
from diffmrc.core.caspt2 import caspt2_contracted_e2_coupled

# h1e_mo, eri_mo: MO integrals in chemist notation, from the CASSCF
# orbitals; ci: reference CI vector; dets: determinant bitmaps.
e2, breakdown = caspt2_contracted_e2_coupled(
    h1e_mo, eri_mo, ci, dets, ncore=ncore, nact=nact, nele=nele)

# With an explicit state-specific generalized Fock matrix, and the
# assembled system for diagnostics:
e2, breakdown, system = caspt2_contracted_e2_coupled(
    h1e_mo, eri_mo, ci, dets, ncore=ncore, nact=nact, nele=nele,
    fock=fock, return_system=True)
```

14.3 Valence-level SOMF for effective-core potentials

The all-electron SOMF operator of `core.somf` uses the bare nuclear charges and the full electron density. With an effective core potential the core electrons are absent, and the spin-orbit part of the ECP replaces their contribution; adding the all-electron formula unchanged would use the ECP effective charge in `int1e_pnucxp` and a valence-only density. The module defines the valence-level operator

$$h_k^{\text{SOMF-ECP}} = \frac{\alpha^2}{2}\left[h_k^{\text{nuc}} + h_k^{2e}[P]\right] - V_k^{\text{ECP}}, \tag{14.4}$$

with h_k^{nuc} the nuclear one-electron term on centers without a spin-orbit ECP, $h_k^{2e}[P]$ the valence $J - \frac{3}{2}K$ mean field of the spin-summed density P ($\mathrm{tr}(PS) =$ `mol.nelectron`), and V_k^{ECP} the three-component `ECPso` integral. On a center carrying a spin-orbit ECP the ECP term replaces the nuclear term: the ECP is fitted to reproduce the all-electron spin-orbit splitting of the valence configuration and already contains the nuclear one-electron term and the core-valence screening for that center. The Breit–Pauli terms carry the factor $\alpha^2/2$; `ECPso` is already an effective operator in energy units and is subtracted.

Entry point	Returns and purpose
`valence_somf_matrix(...)`	The operator, shape `(3, nao, nao)`. Arguments: `(mol, P)`, then `nuclear='auto'`, `ecp_term`, `contract`, `memory_limit`, `convention='i'`, `check`. `nuclear` selects which centers keep the nuclear term (`'auto'` excludes spin-orbit ECP centers); `convention='i'` returns $H_{\mathrm{SO}} = i\sum_k h_k s_k$.
`ecp_soc(mol)`	The three-component `ECPso` integral, shape `(3, nao, nao)`, with no $\alpha^2/2$ factor.
`nuclear_soc(mol, atoms=None)`	The nuclear one-electron SOC in the `int1e_pnucxp` convention, optionally restricted to a list of centers.
`so_ecp_centers(mol)`, `nuclear_centers(mol)`	Atom indices that carry a spin-orbit ECP, and those that do not.
`somf_density_response(...)`	The density-response matrix $M_{rs} = \sum_{pq} D_{pq}[g_{pqrs} - \frac{3}{2}g_{prsq} - \frac{3}{2}g_{sqpr}]$ for `(mol, D)`; only `contract='explicit'` is implemented.
`somf_definition(mol)`	A dictionary recording which centers contribute which term and the approximations that are not fixed by the ECP.

The two-electron kernel is not symmetric under the pair exchange $(pq) \leftrightarrow (rs)$. Consequently the derivative of $\mathrm{tr}(D\,h^{2e}[P])$ with respect to P is not $h^{2e}[D]$; a gradient of an SOC matrix element must use `somf_density_response`, or the equivalent bilinear form. The value layer is NumPy; orbital and CI response are the caller's responsibility, and only the explicit contraction is implemented in the response kernel.

There is no unique first-principles decomposition of an ECP into one- and two-electron spin-orbit parts: the ECP is fitted to atomic reference states, so part of the valence–valence mean field is already folded into its parameters. Adding the explicit valence $J - \frac{3}{2}K$ term is therefore an approximation with a double-counting error that cannot be removed without the ECP generation data. The operator is a prototype for relative comparisons and is not a production absolute-SOC method. The pieces are exposed independently (`nuclear=`, `ecp_term=`) so that variants can be compared and the error bounded.

```
from pyscf import gto

from diffmrc.core import somf_ecp

mol = gto.M(atom='Yb 0 0 0; F 0 0 3.6',
            basis={'Yb': 'stuttgart_rsc', 'F': '6-31g'},
            ecp={'Yb': 'ecpds28mwbso'}, unit='Bohr', verbose=0)

# P: spin-summed density of the electrons present in the calculation,
# trace(P S) = mol.nelectron.  D: the same density, or the transition
# 1-RDM for a matrix element between two scalar states.
h_soc = somf_ecp.valence_somf_matrix(mol, P, convention='i')
m_resp = somf_ecp.somf_density_response(mol, D)
```

On the YbF test molecule the `ECPso` norm on Yb is 17.04 per component, while the raw nuclear operator norm on the same center is of order 10^4; adding both would be dominated by an operator that the ECP was built to replace.

14.4 RIJONX SCF

The reference SCF of the multireference chain still formed the four-center Coulomb tensor through the dense integral path. The RIJONX route fits the Coulomb term and keeps the exchange exact:

$$E = E_{\mathrm{nuc}} + \mathrm{Tr}(Dh) + \tfrac{1}{2}\mathrm{Tr}(DJ_{\mathrm{RI}}) - \tfrac{1}{4}\mathrm{Tr}(DK_{\mathrm{exact}}), \qquad J_{\mathrm{RI}} = WDW^T, \tag{14.5}$$

with $W = BS^{-1/2}$ the metric-folded three-center tensor of Chapter 12. The exchange branch is the dense four-center integral, so the method is RIJONX: not RI-JK, which has no analytic gradient here, and not RIJCOSX, whose fixed-grid prototype is outside the production surface.

Entry point	Returns and purpose
`rijonx_scf(...)`	A converged pyscfad mean-field object; arguments `(mol, auxbasis=None, rcond, **kernel_kwargs)`. `RIJONX_ROHF` is selected when `mol.spin` is non-zero, and a non-converged run raises.
`rijonx_scf_gradient(...)`	Full nuclear gradient of the converged closed-shell energy, shape `(natm, 3)` in Eh/bohr; arguments `(mf, auxbasis, rcond)`.
`rijonx_energy(mol, C, ...)`	The energy of Eq. (14.5) at fixed coefficients.
`rijonx_gradient(mol, C, ...)`	The fixed-coefficient (skeleton) gradient: RI-J analytic, exchange dense, plus core and nuclear terms.
`ri_j_*`, `exact_k_*`, `core_hamiltonian_*`, `nuclear_repulsion_gradient`	The individual energy and gradient pieces.
`make_cosx_grid`, `cosx_exchange_matrix`, `cosx_exchange_energy`	A fixed-grid COSX prototype for exchange; not a production route.

The gradient convention is the point that matters. The functions in `driver.ri_scf` return a fixed-coefficient gradient: the coefficients are held at their reference values and only the integrals and the nuclear repulsion move. That is not the SCF gradient even at a converged density; the missing orbital-response term reduces to the overlap-derivative (energy-weighted density) contribution, $-2\sum_{\mu\in A} Y_{\mu\nu}\,\partial S_{\mu\nu}/\partial R$, with $Y = 2C_{\mathrm{occ}}\epsilon C_{\mathrm{occ}}^T$. Without it the individual terms cancel strongly and the skeleton sum differs from the SCF gradient by order 1 Eh/bohr. `rijonx_scf_gradient` supplies the completion and is the entry point to use for a converged SCF.

The auxiliary basis defaults to `def2-universal-jkfit` and is resolved by `make_auxmol` in `core.ri`, which falls back to the orbital basis when a fitting set does not cover every element. Build a fresh molecule and mean-field object per geometry: the RI factor cache and the pyscfad integral caches are keyed on the built molecule and are not reliably refreshed by an in-place geometry edit.

The exact exchange branch still forms and caches the dense `nao`^{4} tensor and, under geometry differentiation, its dense derivative. RI-J removes the Coulomb part of that work, not the exchange part. The explicit open-shell gradient is not implemented; `rijonx_scf_gradient` refuses fractional or single occupations, and the implicit route `mf.energy_grad()` is the only open-shell path and has not been checked against finite differences. Range-separated Coulomb is refused because the factors are built for the full operator. ECP molecules work when the molecule is built with an explicit `ecp` argument.

```
from diffmrc.driver.ri_scf_driver import rijonx_scf, rijonx_scf_gradient

mf = rijonx_scf(mol, auxbasis='def2-universal-jkfit')
e_tot = mf.e_tot

# Closed-shell full gradient (skeleton plus overlap completion):
grad = rijonx_scf_gradient(mf)

# Open shell: only the implicit path is available.
# grad_open = mf.energy_grad().coords
```

The energy and gradient are validated on H_2O and CO (closed shell), the OH radical (open-shell energy), and HI/def2-svp with an ECP (gradient against central differences). The explicit and implicit closed-shell gradients agree to 10^{-6} Eh/bohr; the measured RIJONX-minus-dense energy difference on HI is -2.6×10^{-5} Eh, which is the J fit, not an implementation error.

14.5 Scalar DF-X2C CASSCF

`core.x2c_casscf_df` adds a state-specific and state-averaged CASSCF layer on the scalar (spin-free) X2C one-electron Hamiltonian with a density-fitted two-electron part:

$$E = \sum_{pq} \gamma_{pq} h_{pq} + \tfrac{1}{2} \sum_{pqrs} \Gamma_{pqrs} (pq|rs) + E_{\mathrm{nuc}}, \qquad (pq|rs) \approx \sum_{P} w_{pqP} w_{rsP}, \tag{14.6}$$

with h the spin-free Foldy–Wouthuysen one-electron Hamiltonian, $w = BS^{-1/2}$ and no picture-change correction on the Coulomb operator. The wavefunction is a real spin-adapted determinant expansion; the orbitals carry no spin-orbit coupling.

Entry point	Returns and purpose
`df_x2c_casscf(...)`	A `DFX2CCASSCFResult` with the converged orbitals, the CI vectors, the state energies, the state-averaged energy and the orbital-gradient norm. Required arguments `(mol, ncore, nact, nele)`; optional auxiliary basis, `nroots`, `weights`, `target`, convergence controls, `theta0` and `rhf`. `nroots=1` is the single-state calculation; `weights` default to equal weights.
`df_x2c_casscf_gradient(...)`	`(grad, parts)` for one CI root, shape `(natm, 3)` in Eh/bohr, with the one-electron, two-electron, Pulay and nuclear contributions separated. Arguments `(mol, result, state=None)`; pass the result object, which carries the CAS space definition and the factor derivatives.
`df_x2c_energies_at(...)`	The state energies at a new geometry with the rotation parameters held fixed; arguments `(mol, result, theta=None)`. The reference orbitals follow the geometry through a displaced SCF and are aligned to `result.c_ref`.
`df_x2c_sa_state_gradient(...)`	The state-specific gradient of a state-averaged calculation; requires `result.nroots >= 2`. Arguments `(mol, result, state=None)`, with finite-difference steps `fd_step` and `theta_step`, a pseudo-inverse cutoff `rcond`, and `return_info`. Returns `(grad, info)` when `return_info` is true.
`df_x2c_rhf(...)`	The closed-shell reference SCF on the same Hamiltonian; arguments `(mol, factors, auxmol, auxbasis, conv_tol, max_cycle, dm0)`.
`sfx2c_hcore(mol)`, `spatial_df_factors(mol, ...)`	The Hamiltonian and the spatial RI factors.

At a converged CASSCF point the orbital and CI responses vanish, so the single-state gradient is the direct derivative at fixed orbitals and CI vector: the one-electron term uses the spin-free X2C hcore derivative, the two-electron term uses the three-center and metric derivatives of Chapter 12, and the Pulay term uses $-\sum_{pq} F_{pq}\, \partial S_{pq}/\partial R$.

A state-averaged calculation is different: the orbitals minimize E_{SA}, so the target-state orbital gradient g_t does not vanish. The state-specific gradient is

$$\frac{dE_t}{dR} = \left.\frac{dE_t}{dR}\right|_{\theta} + z^T M, \qquad Hz = -g_t, \qquad H = \frac{\partial^2 E_{\mathrm{SA}}}{\partial \theta^2}, \tag{14.7}$$

with $M = \partial g_{\mathrm{SA}}/\partial R$. Both H and M are evaluated by finite differences of the analytic state-averaged orbital gradient: H over θ (the CI vectors are re-diagonalized at every step, so the CI response is included) and M over displaced geometries, rebuilding the integrals and the geometry-dependent reference orbitals. The direct term is a central difference of the target-state energy along the same envelope path; the closed-form single-state formula does not apply because the target state is not stationary at the SA orbitals.

The single-state energy and gradient are validated on H_2O/6-31G (6-31G orbital basis, `def2-universal-jkfit`, CAS(2,2), four core spatial orbitals) against a density-fitted sfx2c1e CASSCF in pyscf: the energy agrees within 10^{-5} Eh, the gradient within 10^{-4} relative, the envelope finite difference within 5×10^{-6} Eh/bohr, and the state-averaged state gradients satisfy the weighted sum rule to 10^{-5} Eh/bohr. The layer is scalar (spin-free): full-electron basis sets only, because the spin-free X2C helper raises `NotImplementedError` for ECP molecules. Complex two-component CI, Kramers-restricted response and ECP support are not implemented. The CI, RDM and Fock contractions are dense in the spin-orbital space (`nso`^{4} for the 2-RDM), so the prototype is sized for small validation systems; production f-block active spaces need a factorized 2-RDM route.

```
from diffmrc.core import x2c_casscf_df as xc

res = xc.df_x2c_casscf(mol, ncore=4, nact=2, nele=2,
                       auxbasis='def2-universal-jkfit')
grad, parts = xc.df_x2c_casscf_gradient(mol, res)
e_states = xc.df_x2c_energies_at(mol, res)

res_sa = xc.df_x2c_casscf(mol, ncore=4, nact=2, nele=2, nroots=2,
                          weights=[0.5, 0.5],
                          auxbasis='def2-universal-jkfit')
g_state, info = xc.df_x2c_sa_state_gradient(
    mol, res_sa, state=0, return_info=True)
```

14.6 Validation scope and boundaries

The table collects the validated systems and the boundaries of the five layers. A system absent from the second column is not validated, and the absence is a statement about coverage, not about expected accuracy.

Module	Validated on	Not covered
`nevpt2_zvector`	H_2O/6-31G, LiH/6-31G, CAS(2,2), closed shell, frozen core	open shell, ECP, f block, large basis; the exact treatment covers the `Si` class only
`caspt2` coupled	H_2O/6-31G and CO/6-31G, CAS(2,2), against the reference class table	differentiable coupled path; large active spaces
`somf_ecp`	GeF with a spin-orbit ECP; YbF `ECPso` norms; operator and response identities	production absolute SOC; orbital and CI response
`ri_scf`, `ri_scf_driver`	H_2O, CO, OH (energy), HI/def2-svp with an ECP (gradient)	open-shell analytic gradient; RI-JK; production COSX
`x2c_casscf_df`	H_2O/6-31G, CAS(2,2) and SA(2), against density-fitted sfx2c1e CASSCF	ECP; complex two-component CI; Kramers response; large active spaces